

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Short Answer Test

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA1522

Name	E.B.ASHWINI
Register Number	192511422

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions.* (BL2)

Assessment Tool Weightage: 40%

Dave's Taxonomy: Imitation (Level 1)
Question Count: 5

Total Marks: 50

Code of Conduct

I E.B.ASHWINI (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ASHWINI

Assessment Tasks

Short Answer Test

1. Define cloud computing and explain any four characteristics that make it different from traditional on-premise computing.
2. Trace the evolution of cloud computing from hardware evolution to internet software evolution, and show how these changes enabled modern cloud platforms.
3. Explain the role of server virtualization in cloud computing. Differentiate between virtualization as an enabling technology and cloud computing as a service model.
4. Compare SOAP-based and REST-based web services in terms of communication style, data exchange, and suitability for cloud applications.
5. Differentiate IaaS, PaaS, SaaS, and XaaS with one suitable real-world example for each model.

Short Answer Test Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Accuracy	30%	Accurate definitions and precise explanation of all key terms	Mostly accurate with minor gaps	Partial understanding with noticeable errors	Incorrect or irrelevant explanation
Coverage of Concepts	20%	Covers all expected sub-points completely	Covers most expected points	Covers only basic points	Very limited coverage
Use of Examples	15%	Uses highly	Uses relevant	Uses vague or	No useful

		relevant cloud examples to strengthen the answer	examples with minor mismatch	partially relevant examples	examples
Comparison / Differentiation	20%	Clearly distinguishes concepts with strong logic	Reasonable differentiation with minor overlap	Comparison is weak or incomplete	Fails to differentiate concepts
Clarity of Expression	15%	Clear, organized, and concise answer writing	Generally clear with few issues	Understandable but poorly organized	Difficult to follow

Question 1: Define cloud computing and explain any four characteristics that make it different from traditional on-premise computing.

Cloud computing is a model for delivering computing resources — including servers, storage, databases, networking, software, and analytics tools — over the internet on an on-demand, self-service basis. Instead of purchasing, installing, and maintaining physical hardware and software on-site, organizations and individuals can access these resources remotely from a cloud provider's data centers and pay only for what they use. This shifts computing from a capital-intensive, ownership-based model to a flexible, service-based model.

- **Key characteristics that distinguish cloud computing from traditional on-premise computing:**
- **On-demand self-service** — Resources can be provisioned automatically and instantly whenever a user needs them, without requiring human interaction with the service provider's staff. For example, a developer can spin up a new virtual server in minutes through a web console. In contrast, on-premise computing requires manually ordering hardware, waiting for delivery, and physically installing and configuring it — a process that can take days or weeks.
- **Broad network access** — Cloud services are accessible from anywhere with an internet connection, using standard mechanisms such as web browsers, APIs, and mobile applications, regardless of the client device. On-premise systems, by comparison, are usually confined to an organization's internal network and require VPN access for remote use, which adds complexity and latency.

- **Resource pooling (multi-tenancy)** — Cloud providers pool computing resources — such as processing power, memory, and storage — to serve many customers simultaneously using a multi-tenant model, where physical and virtual resources are dynamically assigned and reassigned according to demand. This allows very efficient use of hardware. On-premise infrastructure, however, is dedicated entirely to a single organization, which often results in servers sitting idle much of the time and wasted capacity.
- **Rapid elasticity and scalability** — Cloud resources can be scaled up or down elastically, and in many cases automatically, to match fluctuating workload demand — sometimes within seconds. This allows applications to handle sudden traffic spikes without manual intervention. On-premise infrastructure has a fixed capacity determined at the time of purchase; scaling up requires buying and physically installing additional hardware, which is slow and expensive, while scaling down wastes the money already spent.
- **Measured service (pay-as-you-go billing)** — Cloud usage is metered in detail (e.g., compute hours, GB of storage, GB of data transferred), and customers are billed only for actual consumption, similar to a utility such as electricity or water. This converts spending from a large upfront capital expenditure (CapEx) into a smaller, predictable, ongoing operational expenditure (OpEx). On-premise computing requires significant upfront investment in hardware and licenses regardless of how much of that capacity is actually used.

Together, these characteristics show that the fundamental difference between cloud and on-premise computing is not just

where the hardware is physically located, but a complete shift in how computing capacity is acquired, scaled, accessed, and paid for.

Question 2: Trace the evolution of cloud computing from hardware evolution to internet software evolution, and show how these changes enabled modern cloud platforms.

- **Mainframe and time-sharing era (1950s–1970s)** — Large, centralized mainframe computers were the dominant form of computing. Because mainframes were extremely expensive, organizations used time-sharing techniques to let multiple users access the same machine simultaneously, each receiving a small slice of processing time. This introduced the foundational idea that computing resources could be shared among many users rather than dedicated to one — a concept central to cloud computing today.
- **Hardware virtualization (1960s–1990s)** — IBM pioneered hardware virtualization, allowing a single physical mainframe to be partitioned into multiple isolated virtual machines, each capable of running its own operating system independently. This decoupled software from the underlying physical hardware for the first time, which is the core technical concept that modern cloud infrastructure still relies on.
- **Client-server and PC era (1980s–1990s)** — The widespread adoption of personal computers and client-server architecture decentralized computing away from mainframes. While this gave organizations more direct control over their own systems, it also meant every company had to build and maintain its own server rooms and IT infrastructure — creating costly, duplicated, and often underutilized infrastructure silos across the industry.
- **Internet and web software evolution (1990s–2000s)** — The rapid growth of the internet, combined with the universal adoption of the web browser as a client, made it possible to

access applications and data remotely rather than only from a local machine. Companies began experimenting with delivering software as a hosted, subscription-based service rather than software installed on individual computers — Salesforce, launched in 1999, is a well-known early example of this software-as-a-service approach.

- **Grid and utility computing (2000s)** — Improvements in bandwidth made it practical to access remote computing resources at scale. Grid computing emerged, allowing organizations to pool distributed computing resources across many machines to solve large computational problems. At the same time, the idea of "utility computing" gained traction — treating computing power as a metered utility that is billed based on consumption, just like electricity — directly foreshadowing modern cloud billing models.
- **Modern cloud platforms (2006–present)** — Amazon launched Amazon Web Services in 2006, starting with EC2 (compute) and S3 (storage), which for the first time commercially combined hardware virtualization, internet-based remote delivery, and metered utility billing into a single, unified, self-service platform. Google App Engine, Microsoft Azure, and many other providers followed, adding higher-level Platform-as-a-Service and Software-as-a-Service layers on top of this infrastructure foundation.

How these threads converged: Hardware virtualization supplied the technical mechanism that allows one physical server to be safely and efficiently shared among many different customers. The evolution of the internet and web software supplied the delivery mechanism, allowing those shared resources to be accessed remotely from

anywhere in the world. Grid and utility computing supplied the business model, establishing that computing power could be metered and billed based on actual consumption rather than sold as a fixed, upfront package. It was the convergence of these three separate historical threads — technology, delivery, and business model — that produced the modern, unified cloud computing platforms that organizations rely on today.

Question 3: Explain the role of server virtualization in cloud computing. Differentiate between virtualization as an enabling technology and cloud computing as a service model.

Server virtualization is the process of using a software layer called a hypervisor (examples include VMware ESXi, Microsoft Hyper-V, Xen, and KVM) to abstract the physical hardware of a server and divide it into multiple independent virtual machines (VMs). Each VM behaves as if it were its own complete computer, with its own operating system, applications, and allocated share of CPU, memory, and storage, even though several VMs are actually running on the same underlying physical hardware, fully isolated from one another.

This capability is central to how cloud computing operates in practice, because it directly enables several of the defining characteristics of the cloud:

- **Resource pooling** — Because a hypervisor keeps each VM isolated, a single physical server can be securely shared among multiple different customers (tenants) at once, without one customer being able to access another's data or affect their performance.
- **Elastic provisioning** — New virtual machines can be created, resized, or deleted within minutes purely through software, without needing to physically install or reconfigure any hardware, which is what makes rapid elastic scaling possible.
- **Better hardware utilization** — Running many VMs on one physical machine dramatically improves how much of that hardware's capacity is actually used, compared to dedicating one physical server to a single, often underused, application.

- **Isolation and fault tolerance** — If one virtual machine crashes, becomes overloaded, or is compromised, the hypervisor keeps the other VMs on the same physical host running normally and unaffected.

Differentiation between virtualization and cloud computing:

Virtualization is a technology that abstracts physical hardware into virtual resources; it operates at the infrastructure/OS level, typically within one data center or server, and its purpose is to improve hardware utilization, isolation, and manageability. A simple example is running five VMs on one physical server using VMware.

Cloud computing, on the other hand, is a service delivery model built on top of that technology; it operates at the business/service level, delivered over the internet, on demand, and billed by usage, with the purpose of delivering computing as a self-service, scalable, metered utility to end users. A simple example is renting a VM instance from AWS EC2 with pay-per-hour billing and automatic elastic scaling.

In short: virtualization is the enabling technology — the "how" — while cloud computing is the service model built using that technology — the "what" is actually delivered to the customer. Virtualization can exist entirely without cloud computing, for example when a company virtualizes its own servers purely for internal use on its own premises. However, modern public cloud computing, at the scale offered by providers like AWS, Azure, or Google Cloud, would not be commercially or technically viable without virtualization underpinning it.

Question 4: Compare SOAP-based and REST-based web services in terms of communication style, data exchange, and suitability for cloud applications.

Web services allow different applications, often built on different platforms and languages, to communicate with each other over a network, most commonly the internet. Two major approaches to building web services are SOAP and REST, and while they achieve similar goals, they differ significantly in design philosophy, structure, and use case.

Communication style:

SOAP (Simple Object Access Protocol) is a strict, formal messaging protocol. It defines a fixed set of rules for how messages must be structured and exchanged, and communication happens by invoking specific, named operations (essentially remote procedure calls) that are formally defined in advance. Every SOAP message is wrapped in a rigid XML envelope, which contains a Header (holding metadata such as authentication or routing information) and a Body (holding the actual message content). Because SOAP is a protocol rather than an architectural style, it enforces a strict contract between client and server that must be followed exactly, leaving little room for flexibility.

REST (Representational State Transfer), by contrast, is not a protocol but an architectural style built directly on top of standard HTTP. It treats every piece of data or functionality as a "resource," each identified by a unique URL, and interactions with that resource are performed using standard HTTP methods: GET (retrieve data), POST (create data), PUT (update data), and DELETE (remove data). This makes REST feel like a natural extension of how the web already

works, rather than an entirely separate messaging system layered on top of it.

Data exchange:

SOAP messages must always be formatted in XML, following a strict schema. The operations a SOAP service supports, along with the exact structure of requests and responses, are formally described in a machine-readable contract called a WSDL (Web Services Description Language) file. This makes SOAP services highly structured and self-describing, but also verbose and rigid — every message carries significant XML overhead even for very simple requests.

REST is far more flexible in its data format. While it can technically exchange XML, plain text, or HTML, in practice the overwhelming majority of REST APIs today use JSON (JavaScript Object Notation), because it is lightweight, easy for both humans and machines to read, and maps naturally onto data structures used in most modern programming languages. This makes REST responses smaller and faster to parse compared to the equivalent SOAP XML message.

State handling:

SOAP is capable of supporting stateful operations, meaning it can maintain context or session information across multiple related calls, which can be useful for complex, multi-step transactions. REST, on the other hand, is stateless by design — every single request must contain all the information the server needs to process it, and the server does not retain any session data between requests. While this means REST cannot maintain conversational context on its own, it is precisely this statelessness that makes REST services easy to scale

horizontally across many servers, since any server can handle any incoming request without needing shared session memory.

Security:

SOAP includes built-in, standardized security extensions, most notably WS-Security, which supports message-level encryption, digital signatures, and formal guarantees around reliability and transactional integrity. This makes SOAP well suited to environments with strict compliance and auditing requirements. REST does not have its own built-in security framework; instead, it relies on security mechanisms provided at the transport layer, primarily HTTPS for encryption in transit, combined with simpler authentication approaches such as API keys, bearer tokens, or OAuth. This is generally sufficient for most web and mobile applications, but offers less formal, message-level guarantees than SOAP's WS-Security.

Performance:

Because every SOAP message must be wrapped in a verbose XML envelope and validated against a strict schema, SOAP messages tend to be significantly larger in size and slower to parse than their REST equivalents. REST, especially when using JSON, produces much smaller, more compact messages, requires less bandwidth, and can be parsed more quickly — all of which matters at cloud scale, where services may

Question 5: Differentiate IaaS, PaaS, SaaS, and XaaS with one suitable real-world example for each model.

Cloud computing services are typically organized into layered service models based on how much of the technology stack the cloud provider manages versus how much the customer must manage themselves. The three foundational models — IaaS, PaaS, and SaaS — form a hierarchy, with a broader category called XaaS extending this same idea to virtually any IT capability delivered as a service.

➤ IaaS — Infrastructure as a Service

IaaS is the most basic and flexible cloud service model. The provider supplies and manages only the fundamental building blocks of computing — physical servers, storage devices, networking hardware, and the virtualization layer that turns physical machines into virtual ones.

Everything above that layer is left entirely to the customer, who must install and manage their own operating system, configure security settings, install middleware, and deploy and maintain their own applications and data. This gives the customer maximum control and flexibility, but also the maximum administrative responsibility, since they are effectively running their own virtual data center.

IaaS is best suited for organizations that need full control over their computing environment, such as companies migrating existing on-premise systems to the cloud without redesigning them. A real-world example is Amazon EC2 (Elastic Compute Cloud), where a user rents virtual machines of a chosen size and then installs and configures the operating system and software exactly as they would on a physical server.

➤ PaaS — Platform as a Service

PaaS builds one layer higher than IaaS. In addition to managing the underlying infrastructure, the provider also manages the operating system, runtime environment, and middleware (such as database engines, development frameworks, and web servers). This means the customer no longer needs to worry about patching operating systems, configuring servers, or managing runtime environments — they simply write their application code and deploy it directly onto the platform.

This significantly speeds up development and reduces operational overhead, though it also means the customer has less low-level control over the underlying environment compared to IaaS. PaaS is best suited for developers who want to focus purely on building and deploying applications without managing infrastructure.

A real-world example is Google App Engine or Heroku, where a developer can push their application code directly to the platform, and the platform automatically handles server provisioning, scaling, and runtime management behind the scenes.

➤ **SaaS — Software as a Service**

SaaS sits at the top of the stack. Here, the provider manages absolutely everything — infrastructure, platform, and the complete, ready-to-use application itself.

The customer does not install, configure, or maintain anything; they simply access the finished software application, typically through a web browser or mobile app, and use it directly.

The only thing the customer manages is their own data and account-level settings within the application. This makes SaaS the easiest model to use, requiring no technical expertise to operate, but it also gives the customer the least control, since they

cannot customize the underlying platform or infrastructure. SaaS is best suited for end users and businesses who simply want to use an application without any involvement in how it is built or hosted. Real-world examples include Gmail for email, Microsoft 365 for office productivity, and Salesforce for customer relationship management — all fully functional applications accessed directly over the internet.

➤ **XaaS — Anything/Everything as a Service**

XaaS is a broader, umbrella term that extends the same "as-a-service" logic beyond the three classic tiers to cover virtually any IT capability that can be packaged and delivered over the cloud. Rather than referring to one specific layer of the stack, XaaS acknowledges that almost any function — databases, security, storage, communication, or even entire desktop environments — can now be offered as an on-demand, subscription-based cloud service rather than something an organization builds and manages itself.

What the provider manages versus what the customer manages varies depending on which specific "X" service is being consumed, since XaaS is a category rather than a single fixed model. Real-world examples include Amazon RDS, which delivers Database as a Service by managing database installation, patching, and backups automatically, and Amazon WorkSpaces, which delivers Desktop as a Service by providing fully managed virtual desktops that users can access remotely.

➤ **Overall Pattern**

The key differentiating logic across these four models is the degree of management responsibility handed over to the cloud provider. As you move from IaaS to PaaS to SaaS, the provider

takes on progressively more of the technical burden — hardware, then platform, then the entire application — while the customer's direct control, configuration effort, and required technical expertise decrease correspondingly at each step.

XaaS does not fit neatly into this single progression; instead, it represents the broader philosophy that essentially any IT function, at any layer, can be reimaged and delivered as an on-demand, subscription-based cloud service, rather than something an organization must build, install, or maintain on its own.

be handling extremely high volumes of requests.